

\$ Aide-mémoire Docker

Les commandes essentielles, groupées par usage. Chaque partie d'une commande garde toujours la même couleur pour qu'on la repère d'un coup d'œil.

■ commande racine ■ sous-commande ■ option / flag ■ valeur à remplacer

Conteneurs et images — les bases

```
docker ps
```

Liste les conteneurs **actifs**, avec pour chacun : ID, image, commande de démarrage, date de création, statut (ex. Up 3 minutes), ports publiés (ex. 0.0.0.0:8082->8082/tcp = port hôte → port conteneur) et nom.

```
docker ps -a
```

Liste **tous** les conteneurs, y compris ceux arrêtés (utile pour les redémarrer).

```
docker start <container>
```

Démarre un conteneur arrêté.

```
docker stop <container>
```

Arrête un conteneur en cours d'exécution.

```
docker rm <container>
```

Supprime un conteneur.

```
docker exec -it <container> bash
```

Ouvre un terminal interactif à l'intérieur d'un conteneur déjà en cours d'exécution (-i = interactif, -t = alloue un pseudo-terminal). Utile pour inspecter le conteneur ou y lancer une commande ponctuelle.

Conseil : les images minimales (ex. basées sur Alpine) n'ont souvent pas bash ; utilise alors sh : `docker exec -it <container> sh`. L'invite affichée (ex. /app #) indique le dossier de travail du conteneur.

```
docker inspect <container> | less
```

Affiche tous les détails d'un conteneur au format JSON (configuration, réseau, montages, variables d'environnement...). Fonctionne aussi sur une image, un volume ou un réseau. | less permet de parcourir une sortie longue page par page.

```
docker exec -it mysql-docker-container bash    # 1. entre dans le conteneur
mysql -u root -p                               # 2. lance le client MySQL (demande le mot de
passe)
```

```
show databases;  
exit  
exit
```

```
# 3. liste les bases présentes  
# 4. quitte le client mysql  
# 5. quitte le shell bash du conteneur
```

Exemple complet : se connecter à une base MySQL qui tourne dans un conteneur, l'interroger, puis ressortir proprement.

```
docker run <image>
```

Crée un nouveau conteneur à partir de l'image et le démarre (télécharge l'image si elle n'est pas déjà en local).

```
docker images
```

Liste les images présentes en local.

```
docker rmi <image_ou_id>
```

Supprime une image.

```
docker pull <image>
```

Télécharge une image depuis un registre (ex. Docker Hub), sans créer de conteneur.

Options avancées de run

```
docker run -p 8082:8082 <image>
```

Publie un port : hôte:conteneur. Le port de gauche est celui exposé sur la machine hôte.

```
docker run --name <nom> <image>
```

Nomme le conteneur au lieu d'un nom aléatoire généré automatiquement. Ce nom sert ensuite directement pour stop, start et rm.

```
docker run -d <image>
```

Lance le conteneur en mode **détaché** (arrière-plan) : le terminal reste libre.

```
docker run -e VAR=valeur
```

Définit une variable d'environnement dans le conteneur. Répétable pour en passer plusieurs (ex. MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD pour une image MySQL).

```
docker run --network host
```

Fait partager au conteneur la pile réseau de la machine hôte : pas d'IP propre, les ports du conteneur sont directement ceux de l'hôte (rend -p inutile).

```
<image> --arg=valeur
```

Ce qui suit le nom de l'image est passé en argument à l'application dans le conteneur, pas à Docker. Ex. `--spring.profiles.active=prod` active un profil Spring Boot.

```
docker run -p 8082:8082 --name mon-conteneur -d mon-image:1.0.0
```

Exemple combinant les options ci-dessus.

```
docker run -e MYSQL_ROOT_PASSWORD=root_password -e MYSQL_DATABASE=my_database -e  
MYSQL_USER=app_user -e MYSQL_PASSWORD=app_password --network host mysql:latest
```

Exemple : lance MySQL avec ses identifiants et sa base initiale, en réseau hôte. MySQL écoute alors directement sur le port 3306 de la machine : une application qui tourne sur l'hôte (ou un autre conteneur en mode host) s'y connecte avec `jdbc:mysql://localhost:3306/my_database`, sans `-p`.

Conseil : si un MySQL tourne déjà sur l'hôte, le port 3306 est occupé et le serveur du conteneur ne pourra pas démarrer. Voir la section Réseaux > Host pour les autres points à connaître sur ce mode.

```
docker rm mon-service  
docker run --name mon-service -p 8082:8082 mon-service:2.0.0 --spring.profiles.active=prod
```

Exemple : remplace un conteneur existant par une nouvelle version de l'image, avec un profil Spring actif.

```
docker run -e SPRING_PROFILES_ACTIVE=<profil>
```

Variable d'environnement reconnue par Spring Boot : active un profil (ex. `test`, `prod`). Équivalent à passer `--spring.profiles.active=<profil>` en argument après l'image — deux façons de faire la même chose.

```
docker build -f Dockerfile-name -t mon-service:2.0.1 .  
docker run --name mon-service -e SPRING_PROFILES_ACTIVE=test -p 8082:8082 mon-service:2.0.1
```

Exemple : builder depuis un Dockerfile spécifique, puis lancer avec un profil de test.

```
docker run -e server.port=<port>
```

Redéfinit une propriété Spring Boot (ici `server.port`) via une variable d'environnement nommée exactement comme la propriété. Spring la reconnaît directement, sans passer par la forme habituelle `SERVER_PORT`. Le port choisi doit correspondre au port conteneur donné à `-p`.

```
docker run --name mon-service -e server.port=8081 -p 8081:8081 -e  
SPRING_PROFILES_ACTIVE=test mon-service:2.0.1
```

Exemple : port et profil définis via des variables d'environnement (le port doit être cohérent avec `-p`).

```
docker run --env-file <fichier>
```

Charge toutes les variables d'environnement depuis un fichier (une par ligne, format `VAR=valeur`), au lieu de les lister une par une avec plusieurs `-e`.

```
docker run --name mon-service -p 8081:8081 --env-file environment.env mon-service:2.0.1
```

Exemple : charger les variables depuis un fichier plutôt que de les passer une à une.

```
docker run --rm
```

Supprime automatiquement le conteneur dès qu'il s'arrête. Évite d'accumuler des conteneurs arrêtés, pratique pour un test ponctuel.

```
docker run --rm alpine
```

Exemple : lance un conteneur jetable, supprimé automatiquement à la sortie.

Build et gestion des images

```
docker build -t nom:version .
```

Construit une image depuis le Dockerfile du dossier courant, et la nomme (tag `nom:version`).

```
docker build -f <Dockerfile> -t nom:version .
```

Précise le fichier à utiliser comme Dockerfile (utile s'il y en a plusieurs, ex. un fichier `Dockerfile-name`). Sans `-f`, Docker cherche un fichier nommé `Dockerfile`.

```
docker tag <image_source> <nom>:<tag>
```

Donne un nouveau nom (et un tag) à une image existante, désignée par son ID ou son nom actuel. Aucune copie n'est faite : les deux noms pointent vers la même image, avec le même `IMAGE ID` dans `docker images`.
Format complet du nom : `[registre/][compte/]nom:tag`.

Conseil : c'est l'étape obligatoire avant de publier une image sur Docker Hub, qui attend un nom de la forme `<compte>/<nom>:<tag>`. Sans `:tag`, Docker utilise `latest`.

```
docker tag 2f7e1deacee7 mon-compte/mon-service:2.0.1
```

Exemple : l'image d'ID `2f7e1deacee7` (repérée avec `docker images`) reçoit le nom `mon-compte/mon-service:2.0.1`, prêt à être envoyé sur le compte Docker Hub `mon-compte`.

```
docker login --username=<compte>
```

Se connecte à Docker Hub (ou à un autre registre, en ajoutant son adresse à la fin). Le mot de passe est demandé ensuite, sans s'afficher à la saisie ; `Login Succeeded` confirme la connexion. Nécessaire avant un `docker push`, ou un `docker pull` depuis un dépôt privé.

Conseil : le message `WARNING! Your password will be stored unencrypted in .../.docker/config.json` signifie que les identifiants sont enregistrés en clair sur la machine. Configure un *credential helper* pour les chiffrer, préfère un jeton d'accès Docker Hub à ton mot de passe (surtout sur un serveur partagé), et pense à `docker logout` quand tu as fini.

```
docker login --username=mon-compte
```

Exemple : connexion au compte Docker Hub `mon-compte`, par exemple sur un serveur distant (une instance cloud) avant d'y récupérer ou d'y publier des images.

```
docker push <compte>/<nom>:<tag>
```

Envoie (publie) une image locale vers un registre, Docker Hub par défaut. Le nom doit commencer par ton compte Docker Hub : c'est pour ça qu'on passe d'abord par `docker tag`. Seules les couches absentes du registre sont envoyées.

Conseil : connecte-toi d'abord avec `docker login`. Un message `denied: requested access to the resource is denied` signifie en général que tu n'es pas connecté, ou que le préfixe du nom ne correspond pas à ton compte. Sur Docker Hub, le dépôt est créé automatiquement au premier push.

```
docker push mon-compte/mon-service:2.0.1
```

Exemple : publie l'image taguée juste au-dessus sur le compte `mon-compte`. Elle pourra ensuite être récupérée sur n'importe quelle machine avec `docker pull mon-compte/mon-service:2.0.1`.

```
docker pull mon-compte/mon-service:2.0.1
```

Exemple : récupère l'image publiée, depuis n'importe quelle machine. Pour une image publique, aucune connexion n'est nécessaire ; pour un dépôt privé, il faut d'abord `docker login`. Cela boucle le cycle `tag` → `push` → `pull`. Si l'image est déjà présente en local avec les mêmes couches, Docker l'indique (`Image is up to date`) sans rien retélécharger.

```
docker run --name mon-service -p 8082:8082 -d mon-compte/mon-service:2.0.1
```

Exemple : sur un serveur distant (ex. une instance cloud), lance l'application directement depuis l'image publiée sur Docker Hub. Si l'image n'est pas encore en local, `docker run` la télécharge d'abord automatiquement. Avec `-d`, Docker affiche seulement l'ID complet du conteneur créé (64 caractères) et rend la main.

Conseil : sur un serveur cloud, ouvre aussi le port 8082 dans le pare-feu (ex. groupe de sécurité) pour accéder à l'application via `http://<IP_publique>:8082`.

```
docker image prune -a
```

Supprime toutes les images non utilisées par un conteneur en une seule fois, après confirmation (`[y/N]`). `-a` et `--all` sont équivalents. Dans le résultat, `untagged` indique un nom d'image retiré (ex. `mysql:9`) et `deleted: sha256:...` une couche effectivement effacée ; `Total reclaimed space` donne l'espace libéré. Lancée juste après `docker container prune`, plus aucun conteneur ne référence les images : elles partent donc toutes, et seront retéléchargées ou reconstruites au prochain `run`.

Précision : sans `-a`, seules les images « dangling » (sans tag) sont supprimées. Avec `-a`, toute image non référencée par un conteneur part aussi — ce n'est pas littéralement « toutes les images » du système.

Filtrage et nettoyage

```
docker ps -a --filter "clé=valeur"
```

Liste les conteneurs en appliquant un filtre. Répéter `--filter` combine les filtres (ET logique).

```
docker ps -a --filter "ancestor=mon-service:2.0.0"
```

Filtre les conteneurs créés à partir de l'image `mon-service:2.0.0`.

```
docker ps -a --filter "status=exited"
```

Filtre les conteneurs arrêtés.

```
... -q | xargs docker rm
```

Ajouté à la suite d'un filtrage : supprime en une fois tous les conteneurs correspondants (-q ne renvoie que les IDs, xargs les passe un à un à docker rm).

```
docker stop $(docker ps -q)
```

Arrête tous les conteneurs en cours d'exécution d'un coup. docker ps -q renvoie uniquement les IDs des conteneurs actifs, et \$(...) les passe en arguments à docker stop, qui affiche l'ID de chaque conteneur arrêté.

Conseil : si aucun conteneur ne tourne, la liste est vide et docker stop affiche une erreur (il attend au moins un argument) : c'est sans conséquence.

```
docker container prune
```

Supprime tous les conteneurs arrêtés en une seule commande, après confirmation (Y/N). Affiche la liste des conteneurs supprimés et l'espace disque libéré (Total reclaimed space). Enchaînée après la commande précédente, elle fait table rase des conteneurs.

Conseil : -f supprime sans demander de confirmation. Les volumes des conteneurs supprimés sont conservés.

```
docker system prune --all
```

Grand nettoyage en une seule commande, après confirmation : supprime les conteneurs arrêtés, les réseaux inutilisés (sauf bridge, host et none), toutes les images non utilisées par un conteneur (grâce à --all ; sans lui, seulement les images sans tag) et le cache de build. Équivaut à enchaîner container prune, network prune, image prune -a et le nettoyage du cache.

Conseil : les volumes ne sont pas touchés ; ajoute --volumes pour supprimer aussi les volumes inutilisés (et donc leurs données). docker system df montre avant coup l'espace occupé par les images, conteneurs, volumes et le cache.

Logs

```
docker logs <container>
```

Affiche les logs du conteneur.

```
docker logs -f <container>
```

Affiche les logs en continu (« follow »), utile pour suivre l'activité en direct.

Volumes

Un volume nommé permet de conserver les données d'un conteneur même s'il est supprimé (contrairement au système de fichiers du conteneur, qui disparaît avec lui). Docker le stocke sur l'hôte, dans

/var/lib/docker/volumes/<nom>/_data.

```
docker volume create <nom>
```

Crée un volume nommé, géré par Docker.

```
docker run -v <volume>:<chemin_conteneur> <image>
```

Monte le volume à l'emplacement indiqué dans le conteneur.

```
docker volume ls
```

Liste tous les volumes existants sur la machine. Un volume nommé apparaît sous son nom (ex. my-app-volume) ; un volume anonyme apparaît sous un identifiant de 64 caractères généré par Docker.

```
docker volume inspect <volume>
```

Affiche le détail d'un volume au format JSON : date de création, driver, point de montage sur l'hôte (Mountpoint), etc. Pour un volume anonyme, le label `com.docker.volume.anonymous` confirme qu'il n'a pas de nom choisi.

```
docker inspect my-app-volume
```

Exemple : `docker inspect` fonctionne aussi directement sur un volume (même résultat que `docker volume inspect`). Le champ Mountpoint indique où sont stockées les données sur l'hôte, ici `/var/lib/docker/volumes/my-app-volume/_data`.

Conseil : avec Docker Desktop (Windows, macOS), ce chemin se trouve dans la machine virtuelle de Docker, pas directement sur ton disque. Pour voir le contenu, passe par un conteneur jetable (voir l'astuce en fin de section).

```
docker volume create my-app-volume
docker run -v my-app-volume:/my-app/data mon-service:2.0.1
```

Exemple : crée le volume my-app-volume puis le monte dans le conteneur.

```
docker run -p 8082:8082 --name mon-service -e SPRING_PROFILES_ACTIVE=test -v my-app-volume:/app/logs mon-service:2.0.1
```

Exemple : attacher un volume existant à un nouveau conteneur. Le volume my-app-volume a déjà été créé (et éventuellement utilisé par un conteneur précédent) : le nouveau conteneur le monte sur `/app/logs` et retrouve les données qu'il contient déjà, puis y ajoute ses propres logs. Ici avec en plus un port publié, un conteneur nommé et le profil Spring test.

Conseil : si le nom de volume donné à `-v` n'existe pas encore, Docker le crée automatiquement (vide) au lieu de signaler une erreur. Vérifie l'orthographe avec `docker volume ls` pour être sûr de réutiliser le bon volume.

```
docker volume rm <volume>
```

Supprime un volume et toutes les données qu'il contient. Accepte un nom ou l'identifiant d'un volume anonyme, et plusieurs volumes à la suite.

Conseil : un volume utilisé par un conteneur, même arrêté, ne peut pas être supprimé. Supprime d'abord le conteneur avec `docker rm`.

```
docker volume rm my-app-volume
```

Exemple : supprime le volume nommé `my-app-volume`. Vérifie ensuite avec `docker volume ls` qu'il n'apparaît plus dans la liste.

Volume nommé en lecture seule

Un volume nommé peut être monté en **lecture seule** : le conteneur peut lire les fichiers, mais ne peut ni les modifier ni les supprimer. Utile pour garantir que des données (typiquement une configuration) ne changent pas pendant l'exécution.

```
docker run -v <volume>:<chemin_conteneur>:ro <image>
```

Le suffixe `:ro` (*read-only*) à la fin de `-v` monte le volume en lecture seule. Sans suffixe, le volume est en lecture-écriture.

```
docker run -d -p 8080:8080 \
-v tomcat-config:/usr/local/tomcat/conf:ro \
-v tomcat-logs:/usr/local/tomcat/logs \
tomcat:latest
```

Exemple avec Tomcat : la configuration (`server.xml`, `tomcat-users.xml`, certificats TLS...) est montée en lecture seule pour éviter toute modification accidentelle ou non autorisée, tandis que le volume des logs reste en écriture pour que Tomcat puisse créer et mettre à jour ses fichiers de log. Le `\` en fin de ligne permet d'écrire une longue commande sur plusieurs lignes.

Conseil : les dossiers `logs`, `temp` et `work` doivent rester en écriture, Tomcat y écrit pendant l'exécution. Évite aussi `:ro` sur webapps : par défaut Tomcat y décompresse les WAR et fait du déploiement automatique. Pour le passer en lecture seule, il faut désactiver `unpackWARs` et l'auto-déploiement, ou déployer l'application déjà décompressée.

Volume anonyme

Si on ne précise que le chemin dans le conteneur (sans nom ni chemin hôte avant les deux-points), Docker crée quand même un volume — mais avec un identifiant généré automatiquement plutôt qu'un nom choisi. Il est donc plus difficile à retrouver et à réutiliser ensuite qu'un volume nommé.

```
docker run -v <chemin_conteneur> <image>
```

Crée un volume anonyme (nom généré par Docker) et le monte à ce chemin.

```
docker run -v /my-app/data mon-service:2.0.1
```

Exemple : Docker crée et gère seul le volume, sans nom choisi.

Bind mount

Autre façon de monter des données : au lieu d'un volume géré par Docker, on relie directement un dossier de la machine hôte au conteneur. C'est vous qui choisissez et gérez cet emplacement (pratique en développement pour accéder directement aux fichiers), contrairement au volume nommé dont l'emplacement est géré par Docker.

```
docker run -v <chemin_hôte>:<chemin_conteneur> <image>
```

Monte un dossier de l'hôte (chemin absolu ou relatif) directement dans le conteneur.

```
docker run -v ~/mon-app/data:/my-app/data mon-service:2.0.1
```

Exemple : monte le dossier ~/mon-app/data de l'hôte dans le conteneur.

```
docker run -p 8082:8082 --name mon-service -e SPRING_PROFILES_ACTIVE=test -v "$(pwd)/my-app-logs:/app/logs" mon-service:2.0.1
```

Exemple : les logs de l'application sont écrits directement dans le dossier my-app-logs du répertoire courant de l'hôte, consultables sans passer par Docker. \$(pwd) est remplacé par le chemin absolu du dossier courant ; les guillemets protègent le chemin s'il contient des espaces. Si le dossier n'existe pas, -v le crée sur l'hôte.

Conseil : \$(pwd) fonctionne sous Linux, macOS et Git Bash. En PowerShell, utilise \${PWD} ; en invite de commandes Windows, %cd%. Sous Git Bash, si le chemin côté conteneur est mal converti (ex. C:/Program Files/Git/app/logs), préfixe la commande par MSYS_NO_PATHCONV=1.

```
ls my-app-logs          # 1. le fichier de log (ex. mon-service.log) est visible sur l'hôte
docker stop mon-service # 2. arrête le conteneur
docker rm mon-service   # 3. supprime le conteneur
ls my-app-logs          # 4. le fichier de log est toujours là
```

Vérification : les fichiers écrits par le conteneur dans un bind mount se trouvent directement dans le dossier de l'hôte et survivent à la suppression du conteneur. Un nouveau conteneur lancé avec le même -v les retrouvera.

Conseil : docker rm -f mon-service arrête et supprime le conteneur en une seule commande.

Bind mount pour un conteneur de base de données

Cas d'usage courant : relier le dossier de données d'une base (ici MySQL) à un dossier de l'hôte, pour que les bases survivent à la suppression du conteneur et restent visibles directement sur l'hôte.

```
docker run --name mon-mysql -e MYSQL_ROOT_PASSWORD=root_password -d -v "$(pwd)/mysql-data:/var/lib/mysql" mysql:9
```

Exemple : persister une base MySQL sur l'hôte. /var/lib/mysql est le dossier où MySQL stocke ses données ; en le reliant au dossier mysql-data du répertoire courant, les bases survivent à la suppression du conteneur et sont retrouvées par un nouveau conteneur lancé avec le même -v.

Conseil : au premier lancement, le dossier mysql-data doit être vide pour que MySQL initialise la base (le mot de passe root n'est pris en compte qu'à ce moment-là). Sous Windows et macOS, un volume nommé (-v mysql-data:/var/lib/mysql) est souvent plus rapide qu'un bind mount pour une base de données.

```
docker exec -it mon-mysql sh # 1. entre dans le conteneur
mysql -u root -p              # 2. se connecte en root (demande le mot de passe)
show databases;               # 3. liste les bases existantes
create database ma_base;      # 4. crée une nouvelle base
exit                           # 5. quitte le client mysql
exit                           # 6. quitte le shell du conteneur
```

Suite de l'exemple : créer une base dans le conteneur MySQL. Elle est écrite dans /var/lib/mysql, donc dans le dossier mysql-data de l'hôte. Après docker rm -f mon-mysql puis le même docker run ... -v "\$(pwd)/mysql-data:/var/lib/mysql" ..., show databases ; affiche toujours ma_base : les données ont survécu au conteneur.

Conseil : docker exec -it mon-mysql mysql -u root -p ouvre directement le client MySQL, sans passer par le shell du conteneur.

```
ls mysql-data # sur l'hôte : un dossier par base + fichiers MySQL
```

Vérification côté hôte : le dossier `mysql-data` contient un sous-dossier par base (`ma_base`, `mysql`, `sys`, `performance_schema`) ainsi que les fichiers internes de MySQL (`ibdata1`, `binlog.*`, `undo_*`, certificats `*.pem...`). Relancer ensuite le même `docker run ... -v "$(pwd)/mysql-data:/var/lib/mysql" mysql:9` redémarre MySQL sur ces données existantes.

Conseil : ne modifie pas ces fichiers à la main, et ne partage pas le même dossier entre deux conteneurs MySQL lancés en même temps : la base serait corrompue.

tmpfs mount (Linux uniquement)

Monte un dossier directement en mémoire vive de l'hôte : rien n'est jamais écrit sur disque, les données disparaissent à l'arrêt du conteneur et ne sont partagées ni avec l'hôte ni avec d'autres conteneurs. Utile pour des fichiers temporaires ou sensibles (logs éphémères, caches).

```
docker run --tmpfs <chemin_conteneur> <image>
```

Monte le chemin donné en mémoire (tmpfs), Linux uniquement.

```
docker run -p 8082:8082 --name mon-service --tmpfs /app/logs mon-service:2.0.1
```

Exemple : l'application tourne normalement (port publié, conteneur nommé), mais ses logs dans `/app/logs` vivent en mémoire, jamais sur disque. Ils disparaissent dès que le conteneur s'arrête.

Conseil : « Linux uniquement » concerne les conteneurs Linux. Avec Docker Desktop sous Windows ou macOS, --tmpfs fonctionne aussi, car les conteneurs tournent dans une machine virtuelle Linux.

```
docker ps                # 1. vérifie que le conteneur tourne
docker inspect mon-service | less # 2. cherche HostConfig.Tmpfs dans le JSON
docker exec -it mon-service sh # 3. entre dans le conteneur
mount | grep /app/logs      # 4. affiche « tmpfs on /app/logs type tmpfs »
exit                        # 5. quitte le conteneur
```

Vérification : dans la sortie de `docker inspect` (parcourue avec `less`), la section `HostConfig` contient `"Tmpfs": { "/app/logs": "" }`, ce qui confirme que ce chemin est monté en mémoire. À l'intérieur du conteneur, `mount` liste tous les points de montage : la ligne `tmpfs on /app/logs type tmpfs (rw,nosuid,nodev,noexec,relatime)` le confirme aussi. Les options entre parenthèses sont celles appliquées par défaut par `--tmpfs` : lecture-écriture (`rw`), mais aucun programme ne peut être exécuté depuis ce dossier (`noexec`). Sans `grep`, la liste est longue (`systeme`, `/proc`, `/sys...`).

Conseil : `docker inspect -f '{{json .HostConfig.Tmpfs}}' mon-service` affiche uniquement ce champ, sans avoir à parcourir tout le JSON. Pour limiter la mémoire utilisée, précisez une taille : `--tmpfs /app/logs:size=64m`.

Astuce : inspecter le contenu d'un volume

On peut monter un volume existant dans un conteneur jetable, juste pour regarder ce qu'il contient, sans toucher au conteneur qui l'utilise habituellement.

```
docker run --rm -it -v my-app-volume:/data alpine sh
```

`--rm` supprime ce conteneur temporaire à la sortie, `-it` ouvre un shell interactif, et `alpine` est une image minimale pratique pour ce genre d'exploration ponctuelle. Une fois dedans, `ls /data` montre le contenu du volume.

Réseaux

Le réseau d'un conteneur dépend de son **driver** réseau, choisi avec `--network` au lancement (ou `-d` à la création

d'un réseau). Docker en propose cinq principaux, selon que les conteneurs tournent sur une seule machine ou plusieurs, et selon le niveau d'isolation voulu.

```
docker network ls
```

Liste les réseaux existants, avec leur ID, leur nom (NAME), leur driver (DRIVER) et leur portée (SCOPE). Par défaut, Docker en crée trois : bridge, host et none (driver null, puisqu'il n'y a pas de réseau). Un réseau créé avec `docker network create` apparaît à côté, ex. my-net avec le driver bridge. La portée local signifie que le réseau n'existe que sur cette machine (un réseau overlay aurait la portée swarm).

Bridge

Sur le réseau bridge par défaut : **chaque conteneur reçoit une adresse IP privée** ; les conteneurs peuvent se parler entre eux, mais **uniquement par adresse IP** (pas par nom) ; et pour être joignable depuis l'extérieur du réseau bridge (l'hôte, une autre application), un conteneur doit **publier un port** avec -p.

```
docker run --network bridge <image>
```

Réseau par défaut : plusieurs conteneurs tournent sur la même machine, dans un réseau privé isolé de l'hôte. Pour y accéder depuis l'hôte, on publie des ports avec -p. Sans --network, c'est ce driver qui est utilisé.

Conseil : sur un réseau bridge créé soi-même (`docker network create <réseau>`), les conteneurs se joignent par leur nom (DNS intégré), ce qui n'est pas le cas sur le bridge par défaut.

```
docker run -d --name produits -p 8081:8080 produits:1.0
```

```
docker run -d --name commandes -p 8082:8080 commandes:1.0
```

Exemple : deux applications sur le même hôte, toutes deux sur le réseau bridge par défaut. Chacune écoute sur le port 8080 dans son conteneur, mais elles sont publiées sur des ports hôte différents (8081 et 8082), car deux conteneurs ne peuvent pas publier le même port hôte.

- Entre conteneurs : commandes appelle produits via son IP privée et le port conteneur, ex.

`http://<IP_produits>:8080/products.`

- Depuis l'extérieur : une autre application passe par l'hôte et le port publié, ex. `http://<hôte>:8082` pour commandes.

- Plusieurs instances d'une même image peuvent tourner côte à côte : chacune a sa propre IP et son propre port hôte.

```
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' <container>
```

Affiche l'adresse IP privée d'un conteneur sur son réseau (ex. 172.17.0.2), celle à utiliser pour l'appeler depuis un autre conteneur du bridge par défaut.

Conseil : cette IP peut changer à chaque redémarrage du conteneur. Pour des appels stables entre conteneurs, préfère un réseau bridge personnalisé où l'on appelle les conteneurs par leur nom.

```
docker run --name mon-mysql -e MYSQL_DATABASE=ma_base -e MYSQL_ROOT_PASSWORD=root_password  
-e MYSQL_USER=app_user -e MYSQL_PASSWORD=app_password mysql/mysql-server:latest
```

Exemple : une base MySQL sur le bridge par défaut (aucun --network). Au démarrage, elle crée la base ma_base et l'utilisateur app_user. Sans -p, le port MySQL (3306) n'est pas accessible depuis l'hôte : seuls les autres conteneurs du bridge peuvent la joindre, via son IP privée, ex. `jdbc:mysql://<IP_mon-mysql>:3306/ma_base`. Dans `docker ps`, la colonne PORTS affiche 3306/tcp, 33060-33061/tcp sans 0.0.0.0:--> : ces ports sont exposés par l'image mais non publiés sur l'hôte. Pour trouver l'IP, `docker inspect mon-mysql` : champ IPAddress dans NetworkSettings.Networks.bridge (ou la commande `inspect -f ci-dessus`). Sans -d, le conteneur reste attaché au terminal et affiche ses logs.

Conseil : mysql/mysql-server est l'image publiée par Oracle, qui n'est plus maintenue ; pour un nouveau projet, préfère

l'image officielle `mysql` (mêmes variables d'environnement).

```
docker run --name mon-service \
  -e server.port=8081 -p 8081:8081 \
  -e spring.datasource.username=app_user \
  -e spring.datasource.password=app_password \
  -e spring.datasource.url=jdbc:mysql://172.17.0.2:3306/ma_base \
  -e SPRING_PROFILES_ACTIVE=prod \
  mon-service:2.0.1
```

Exemple : démarrer une application Spring Boot sur le bridge par défaut, connectée à la base MySQL ci-dessus. Les variables `spring.datasource.*` remplacent la configuration de `application.properties` : identifiants (`app_user` / `app_password`, ceux créés par `MYSQL_USER` / `MYSQL_PASSWORD`) et URL JDBC pointant sur l'IP privée du conteneur MySQL (`172.17.0.2`, obtenue avec `docker inspect`) et son port conteneur `3306`. L'application, elle, est publiée sur l'hôte via `-p 8081:8081`, en cohérence avec `server.port`.

Conseil : si le conteneur MySQL redémarre, son IP peut changer et l'URL ne fonctionnera plus. Sur un réseau bridge personnalisé, on écrirait simplement `jdbc:mysql://mon-mysql:3306/ma_base`.

Bridge personnalisé

Plutôt que le bridge par défaut, on peut créer son propre réseau bridge. Les conteneurs qui y sont rattachés sont isolés des autres réseaux et, surtout, **se joignent par leur nom de conteneur** grâce au DNS intégré de Docker : plus besoin de chercher leur IP.

```
docker network create <r  seau>
```

Cr  e un r  seau. Sans `-d`, le driver utilis   est `bridge`. Docker affiche en retour l'identifiant (64 caract  res) du r  seau cr   .

```
docker network create my-net
```

Exemple : cr  e le r  seau bridge `my-net`. Il appara  t ensuite dans `docker network ls`    c  t   de `bridge`, `host` et `none`. Les conteneurs lanc  s avec `--network my-net` pourront s'appeler par leur nom (ex. `jdbc:mysql://mon-mysql:3306/ma_base`).

```
docker network inspect <r  seau>
```

Affiche le d  tail d'un r  seau au format JSON : `Driver`, `Scope`, plage d'adresses attribu  e (IPAM.Config : `Subnet` et `Gateway`), et `Containers` : la liste des conteneurs connect  s avec leur adresse IP sur ce r  seau.

```
docker network inspect my-net
```

Exemple : inspecte le r  seau `my-net`. Juste apr  s sa cr  ation, `Containers` est vide ; le `Subnet` montre la plage d'IP propre    ce r  seau (ex. `172.18.0.0/16`), distincte de celle du bridge par d  faut (`172.17.0.0/16`).

Conseil : `docker network inspect -f '{{json .Containers}}' my-net` n'affiche que les conteneurs connect  s.

```
docker network connect <r  seau> <container>
```

Connecte un conteneur d  j   existant (m  me en cours d'ex  cution, sans le red  marrer)    un r  seau suppl  mentaire. Le conteneur reste aussi sur ses r  seaux actuels : il a alors une IP sur chacun.

Conseil : `docker network disconnect <r  seau> <container>` fait l'inverse et retire le conteneur d'un r  seau.

```
docker network connect my-net mon-mysql
```

Exemple : rattache le conteneur MySQL (lancé plus haut sur le bridge par défaut) au réseau my-net. Les conteneurs de my-net peuvent désormais le joindre par son nom : jdbc:mysql://mon-mysql:3306/ma_base. docker network inspect my-net le montre dans Containers.

```
docker run --network my-net --name mon-service \
-e server.port=8081 -p 8081:8081 \
-e spring.datasource.username=app_user \
-e spring.datasource.password=app_password \
-e spring.datasource.url=jdbc:mysql://mon-mysql:3306/ma_base \
mon-service:2.0.1 --spring.profiles.active=prod
```

Exemple : l'application Spring Boot lancée directement sur my-net (--network my-net), où se trouve aussi mon-mysql. L'URL JDBC utilise le nom du conteneur MySQL au lieu de son IP : jdbc:mysql://mon-mysql:3306/ma_base, résolu par le DNS de Docker. Elle continue donc de fonctionner même si l'IP de MySQL change. Le profil est ici passé en argument après l'image (--spring.profiles.active=prod), équivalent de -e SPRING_PROFILES_ACTIVE=prod.

```
docker exec mon-service ping -c 4 mon-mysql
```

Vérification : depuis le conteneur de l'application, envoie 4 pings (-c 4) au conteneur MySQL par son nom. La première ligne de réponse, ex. PING mon-mysql (172.18.0.2), montre que le DNS de Docker a résolu le nom en une IP du réseau my-net (plage 172.18.x.x) ; 0% packet loss confirme que les deux conteneurs communiquent. Sans -it, docker exec lance une commande ponctuelle et affiche simplement son résultat.

Conseil : ping n'est pas installé dans toutes les images (surtout les images minimales) ; si la commande est introuvable, utilise un conteneur jetable busybox sur le même réseau (voir ci-dessous).

```
docker run --rm --network my-net busybox ping -c 4 mon-mysql
```

Vérification avec un conteneur jetable : lance un conteneur busybox (image minuscule qui contient ping, nslookup, wget...) sur my-net, qui ping mon-mysql par son nom, puis est supprimé automatiquement (--rm). Pratique quand l'image de l'application n'a pas d'outils réseau. Au premier lancement, le message Unable to find image 'busybox:latest' locally est normal : Docker télécharge l'image puis exécute la commande.

Host

Réservé aux **conteneurs Linux**. Le conteneur utilise directement le réseau de la machine hôte : il n'a ni réseau séparé ni adresse IP propre (il utilise celle de l'hôte), et il n'y a **pas besoin de publier de port** : un port ouvert par l'application est directement un port de l'hôte.

```
docker run --network host <image>
```

Le conteneur partage directement le réseau de la machine hôte : pas d'IP propre, ses ports sont ceux de l'hôte, donc -p est inutile. Voir aussi l'exemple MySQL dans « Options avancées de run ».

```
docker run -d --name produits --network host produits:1.0 # 8080
docker run -d --name commandes --network host -e server.port=8081 commandes:1.0 # 8081
```

Exemple : deux applications en réseau host, sans aucun -p. Comme elles partagent les ports de l'hôte, elles doivent écouter sur des ports différents (ici 8081 via server.port pour l'application Spring Boot).

- Depuis une autre application sur l'hôte : http://localhost:8080/products et http://localhost:8081/orders.
- Depuis une autre machine : http://<hôte>:8081, directement sur le port de l'application.

Conseil : en mode host, `-p` est ignoré, et si deux conteneurs (ou un conteneur et un programme de l'hôte) veulent le même port, le second ne pourra pas démarrer son serveur. Avec Docker Desktop (Windows, macOS), ce mode ne se comporte pas comme sous Linux ; les versions récentes le proposent en option à activer dans les paramètres.

```
docker run --network host --name mon-service \
-e server.port=8081 \
-e spring.datasource.username=app_user \
-e spring.datasource.password=app_password \
-e spring.datasource.url=jdbc:mysql://localhost:3306/my_database \
mon-service:2.0.1 --spring.profiles.active=prod
```

Exemple : application Spring Boot et MySQL (lancé en mode host, voir « Options avancées de run ») tous deux en réseau host. Comme ils partagent le réseau de la machine, l'application joint la base via `localhost:3306` : pas d'IP à chercher, pas de nom de conteneur, pas de `-p`. L'application est directement accessible sur `http://localhost:8081`.

Joindre l'hôte depuis un conteneur : `host.docker.internal`

Depuis un conteneur, `localhost` désigne le conteneur lui-même, pas la machine hôte. Pour joindre un service qui écoute sur l'hôte (une base installée localement, ou un conteneur lancé en mode host), on utilise le nom spécial `host.docker.internal`, qui pointe vers l'hôte.

```
docker run --name mon-service \
-e server.port=8081 -p 8081:8081 \
-e spring.datasource.username=app_user \
-e spring.datasource.password=app_password \
-e spring.datasource.url=jdbc:mysql://host.docker.internal:3306/my_database \
-e SPRING_PROFILES_ACTIVE=prod \
mon-service:2.0.1
```

Exemple : l'application tourne sur le bridge par défaut (publiée avec `-p 8081:8081`), tandis que MySQL écoute sur le port 3306 de l'hôte (MySQL en mode host, ou installé directement sur la machine). L'URL JDBC `jdbc:mysql://host.docker.internal:3306/my_database` sort du conteneur pour atteindre l'hôte.

Conseil : `host.docker.internal` est disponible automatiquement avec Docker Desktop (Windows, macOS). Sous Linux, ajoute `--add-host=host.docker.internal:host-gateway` au `docker run` pour le définir.

None

```
docker run --network none <image>
```

Aucun réseau : le conteneur n'a que l'interface locale (`localhost`) et ne peut joindre ni l'hôte, ni Internet, ni d'autres conteneurs. Utile pour un traitement isolé qui n'a besoin d'aucun accès réseau.

Overlay

```
docker network create -d overlay <réseau>
```

Relie des conteneurs qui tournent sur des machines différentes, comme s'ils étaient sur le même réseau.

Conseil : ce driver s'utilise avec Docker Swarm ; il faut d'abord initialiser le cluster avec `docker swarm init`.

Macvlan

```
docker network create -d macvlan --subnet <sous-réseau> --gateway <passerelle> -o parent=
```

<interface> <réseau>

Le conteneur reçoit sa propre adresse MAC et sa propre IP : il apparaît comme une machine distincte sur le réseau physique. Utile pour des applications qui doivent être jointes directement sur le réseau local.

Conseil : fonctionne sur un hôte Linux. Par défaut, l'hôte lui-même ne peut pas joindre ses conteneurs macvlan.

Docker Compose

Docker Compose décrit **toute une application multi-conteneurs dans un seul fichier YAML** (docker-compose.yaml) : base de données, serveur de configuration, discovery, gateway, microservices... Pour chaque service, on y déclare ce qu'on passerait sinon à `docker run` : nom, ports, variables d'environnement, volumes, réseau. Compose peut ensuite construire les images (build), lancer l'ensemble (run) et publier les images (publish) en une commande.

```
services:
  mysql:           # base de données
  config-server:   # serveur de configuration
  eureka:          # annuaire de services (discovery)
  gateway:         # API Gateway
  users:           # microservice Users
  products:        # microservice Products
```

Structure d'un fichier docker-compose.yaml : sous `services:`, une entrée par conteneur. Chaque service reçoit ensuite sa configuration (image ou build, ports, environnement, volumes, networks...). Le fichier se crée à la racine du projet ; Compose reconnaît automatiquement les noms `compose.yaml` (nom recommandé aujourd'hui), `compose.yml`, `docker-compose.yaml` et `docker-compose.yml`. Pour un autre nom, voir l'option `-f` ci-dessous.

Conseil : Compose crée automatiquement un réseau bridge pour le projet ; les services s'y joignent par leur nom de service (ex. `jdbc:mysql://mysql:3306/...`), comme sur un bridge personnalisé.

```
services:
  mysql:
    container_name: mysql-server
#mon-service:
```

`container_name` fixe le nom du conteneur, comme `--name` avec `docker run`. Sans lui, Compose génère un nom de la forme `<projet>-<service>-<numéro>`, où le projet est par défaut le nom du dossier qui contient le fichier : ex. `users-mysql-1`. Ici, le conteneur s'appellera `mysql-server`. Une ligne commençant par `#` est un commentaire YAML (service désactivé).

Conseil : le nom du service (`mysql`) reste celui à utiliser pour joindre le conteneur depuis les autres services. Et avec un `container_name` fixe, on ne peut pas lancer plusieurs instances du service (`--scale`), car deux conteneurs ne peuvent pas porter le même nom. Attention aussi à l'indentation : YAML n'accepte que des espaces, pas de tabulations.

```
services:
  mysql:
    container_name: mysql-server
    image: mysql:latest
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ma_base
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
#mon-service:
```

image indique l'image à utiliser (équivalent du dernier argument de `docker run`), et `environment` liste les variables d'environnement du conteneur (équivalent des `-e`), une par ligne au format `CLÉ: valeur`. La syntaxe `${MYSQL_PASSWORD}` est remplacée par `Compose` au lancement, avec la valeur de la variable du même nom prise dans le terminal ou dans un fichier `.env` placé à côté du `docker-compose.yaml` (ex. une ligne `MYSQL_PASSWORD=app_password`). Les valeurs fixes, comme `ma_base`, s'écrivent directement.

Conseil : ce mécanisme évite d'écrire les mots de passe en clair dans le fichier `Compose` ; ajoute `.env` au `.gitignore` pour ne pas le committer. `docker compose config` affiche le fichier final, variables remplacées, pour vérifier. Une variable absente est remplacée par une chaîne vide (avec un avertissement) ; `${VAR:-défaut}` fournit une valeur par défaut.

```
services:
  mysql:
    container_name: mysql-server
    image: mysql:latest
    environment: # ...
    ports:
      - "3306:3306"
#mon-service:
```

`ports` publie des ports du conteneur sur l'hôte, comme `-p` avec `docker run`, au format `"<port hôte>:<port conteneur>"`. Ici, MySQL devient joignable depuis la machine sur `localhost:3306`. `ports` est une liste `YAML` : chaque port s'écrit sur sa propre ligne, précédé d'un tiret.

Conseil : garde les guillemets autour de `"3306:3306"`, car `YAML` peut mal interpréter certaines valeurs contenant `:`. Inutile de publier un port pour que deux services du même fichier se parlent : ils se joignent déjà par leur nom (`mysql:3306`). Si le port `3306` est déjà pris sur la machine, change seulement le port hôte : `"3307:3306"`.

```
services:
  mysql:
    image: mysql:latest      # + container_name, environment, ports...
    volumes:
      - mysql-data:/var/lib/mysql
#mon-service:
```

```
volumes:
  mysql-data:
```

Persiste les données de MySQL avec un volume nommé. Deux niveaux : dans le service, `volumes` monte le volume `mysql-data` sur `/var/lib/mysql` (équivalent de `-v mysql-data:/var/lib/mysql`) ; en bas du fichier, la section `volumes` de premier niveau déclare ce volume nommé, que `Compose` crée au premier `up`. Comme `ports`, c'est une liste `YAML` (un tiret par montage).

Conseil : `Compose` préfixe le nom du volume par celui du projet (ex. `<projet>_mysql-data` dans `docker volume ls`). Le volume survit à `docker compose down`, mais `down -v` le supprime avec ses données. Pour un `bind mount`, écris un chemin à la place du nom (ex. `./mysql-data:/var/lib/mysql`) : aucune déclaration en bas de fichier n'est alors nécessaire.

```
services:
  mysql:                                     # ...

  mon-service:
    container_name: mon-service
    build:
      context: "./mon-service"
      dockerfile: "Dockerfile-multi-stage"
      tags:
```

```
- mon-service:latest
```

`build` remplace `image` quand l'image doit être construite à partir d'un Dockerfile au lieu d'être téléchargée. `context` est le dossier envoyé au build (chemin relatif au fichier Compose, ici le dossier du microservice) et `dockerfile` le nom du Dockerfile à utiliser dans ce dossier. C'est l'équivalent de `docker build -f Dockerfile-multi-stage ./mon-service`. Sans `dockerfile`, Compose cherche un fichier nommé `Dockerfile`. `tags` (une liste YAML) ajoute des noms à l'image construite, ici `mon-service:latest`, utilisable ensuite avec `docker run` OU `docker push`.

Conseil : sans `image`, l'image construite reçoit aussi le nom automatique `<projet>-<service>`, en plus des noms listés dans `tags` ; pour choisir le nom principal, ajoute `image: mon-compte/mon-service:2.0.1` à côté de `build`. `docker compose up` réutilise l'image déjà construite : après une modification du code, relance avec `docker compose up --build` (OU `docker compose build`) pour la reconstruire.

```
mon-service:
  container_name: mon-service
  image: mon-compte/mon-service:2.0.1
  build:
    context: "./mon-service"
    dockerfile: "Dockerfile-multi-stage"
  tags:
    - mon-service:latest
    - mon-service:2.0.1
```

Exemple : `image` et `build` ensemble. Compose construit l'image à partir du Dockerfile et lui donne comme nom principal la valeur de `image`, ici au format Docker Hub `<compte>/<nom>:<tag>`, plus les noms locaux listés dans `tags` (`mon-service:latest` et `mon-service:2.0.1`). Tous pointent vers la même image (même IMAGE ID), comme avec `docker tag`.

Conseil : avec un nom au format Docker Hub, `docker compose push` publie directement l'image du service sur ton compte (après `docker login`), sans passer par `docker tag`.

```
services:
  mysql:                                # ...

  mon-service:
    image: mon-compte/mon-service:2.0.1
    build:                                # ...
    depends_on:
      - mysql
```

`depends_on` définit l'ordre de démarrage : Compose lance d'abord `mysql`, puis `mon-service` qui en dépend (et les arrête dans l'ordre inverse avec `down`). C'est une liste YAML de noms de services. Lancer `docker compose up mon-service` démarre aussi automatiquement `mysql`.

Conseil : par défaut, Compose attend seulement que le conteneur MySQL soit démarré, pas que la base soit prête à accepter des connexions ; l'application Spring Boot peut donc échouer à se connecter pendant les premières secondes. Pour attendre vraiment, ajoute un `healthcheck` au service `mysql` et utilise la forme longue `depends_on: mysql: condition: service_healthy`.

```
mon-service:
  image: mon-compte/mon-service:2.0.1
  build:                                # ...
  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/ma_base
    SPRING_DATASOURCE_USERNAME: ${MYSQL_USER}
    SPRING_DATASOURCE_PASSWORD: ${MYSQL_PASSWORD}
```

```
SPRING_PROFILES_ACTIVE: prod
depends_on:
  mysql:
    condition: service_healthy
```

Exemple : le microservice Spring Boot configuré dans Compose. Les variables `SPRING_DATASOURCE_*` remplacent la configuration de `application.properties` ; l'URL JDBC utilise le **nom du service** `mysql` comme hôte (réseau Compose, comme sur un bridge personnalisé), et les identifiants réutilisent les mêmes variables `${MYSQL_USER}` / `${MYSQL_PASSWORD}` que le service MySQL. `depends_on` est ici sous sa **forme longue** : `condition: service_healthy` fait attendre que MySQL soit réellement prêt avant de démarrer l'application.

Conseil : `service_healthy` exige un `healthcheck` sur le service `mysql`, sinon Compose refuse de démarrer. Exemple ci-dessous.

```
services:
  mysql:
    image: mysql:latest      # + environment, ports, volumes...
    healthcheck:
      test: ["CMD-SHELL", "mysql -h 127.0.0.1 -u $$MYSQL_USER -p$$MYSQL_PASSWORD -e 'SELECT 1'"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s
```

`healthcheck` définit une commande que Docker exécute régulièrement **dans le conteneur** pour savoir s'il est en bonne santé. Ici, `CMD-SHELL` lance la commande via le shell du conteneur : le client `mysql` se connecte et exécute `SELECT 1`, ce qui ne réussit que lorsque MySQL accepte vraiment les connexions. Le conteneur passe alors à l'état `healthy` (visible dans `docker ps`), ce qu'attend `condition: service_healthy`. Les réglages : `interval` = délai entre deux tests (30 s par défaut), `timeout` = durée maximale d'un test, `retries` = nombre d'échecs consécutifs avant l'état `unhealthy`, `start_period` = délai de démarrage pendant lequel les échecs ne sont pas comptés (le temps que MySQL s'initialise).

Conseil : le `$$` est volontaire : il empêche Compose de remplacer la variable au lancement, et le shell du conteneur la lit dans son propre environnement (`MYSQL_USER`, `MYSQL_PASSWORD`).

```
export MYSQL_ROOT_PASSWORD=root_password      # Linux, macOS, Git Bash
export MYSQL_USER=app_user
export MYSQL_PASSWORD=app_password
$env:MYSQL_ROOT_PASSWORD="root_password"      # PowerShell (idem pour les autres)
docker compose up                             # lance les services avec ces valeurs
```

Définit dans le terminal une variable que Compose utilisera pour remplacer `${MYSQL_ROOT_PASSWORD}`. La syntaxe dépend du shell : `export` sous Linux, macOS et Git Bash ; `$env:` en PowerShell (valeur entre guillemets). Ici, les trois variables référencées dans le fichier Compose ci-dessus sont définies une par une, puis on lance `docker compose up` **depuis ce même terminal**.

Conseil : ces variables ne vivent que le temps de la session du terminal ; en fermant la fenêtre, il faut les redéfinir. Pour ne pas les retaper à chaque fois, utilise plutôt un fichier `.env` (hors Git).

```
docker compose -f <fichier> <commande>
```

Indique explicitement le fichier Compose à utiliser, quand il ne porte pas un des noms reconnus par défaut ou quand on veut en choisir un parmi plusieurs. L'option se place **avant** la sous-commande (`up`, `down`...).

Conseil : `-f` peut être répété pour combiner plusieurs fichiers, le second complétant ou surchargeant le premier (ex. `-f compose.yaml -f compose.prod.yaml`).

```
docker compose -f docker-compose-prod.yaml up -d
```

Exemple : lance en arrière-plan les services décrits dans un fichier dédié à la production. Pour les arrêter, il faut repasser le même fichier : `docker compose -f docker-compose-prod.yaml down`.

```
docker compose up
```

Crée et démarre tous les services décrits dans le fichier (réseau, volumes, conteneurs), en téléchargeant ou construisant les images si nécessaire. À lancer depuis le dossier qui contient `docker-compose.yaml`.

Conseil : ajoute `-d` pour lancer en arrière-plan, comme avec `docker run`. L'ancienne syntaxe `docker-compose up` (avec un tiret) fonctionne encore sur certaines installations.

```
docker compose up -d --build
```

La commande du quotidien pendant le développement : `--build` **reconstruit** les images des services qui ont une section `build` (pour prendre en compte les dernières modifications du code), puis `-d` démarre l'ensemble **en arrière-plan** et rend la main au terminal. Les services basés sur une image téléchargée (ex. `mysql`) ne sont pas reconstruits.

Conseil : en mode `-d`, suis les logs avec `docker compose logs -f` (OU `docker compose logs -f <service>` pour un seul service), et vérifie l'état des conteneurs avec `docker compose ps`.

```
docker compose up -d --pull always
```

Démarre les services en arrière-plan en **retéléchargeant d'abord les images** depuis le registre, même si elles sont déjà présentes en local. Utile pour récupérer la dernière version publiée d'une image au même tag (ex. `mysql:latest`, OU `mon-compte/mon-service:2.0.1` republiée sur Docker Hub), typiquement sur un serveur de déploiement.

Conseil : les valeurs possibles sont `always` (toujours télécharger), `missing` (valeur par défaut : seulement si l'image est absente) et `never` (n'utiliser que les images locales). Pour un service avec `build`, c'est plutôt `--build` qui sert à prendre en compte le code local.

```
docker compose build
```

Construit **uniquement** les images des services qui ont une section `build`, sans créer ni démarrer de conteneur. Les images reçoivent les noms définis par `image` et `tags`. Pratique pour vérifier que le build passe, ou pour préparer les images avant un `docker compose push` ; un `docker compose up` lancé ensuite les utilisera directement.

Conseil : `docker compose build <service>` ne construit qu'un service.

```
docker compose build --no-cache
```

Reconstruit les images **entièrement**, sans réutiliser les couches mises en cache lors des builds précédents : chaque instruction du Dockerfile est réexécutée (téléchargement des dépendances Maven, compilation...). La sortie indique le nombre d'images à construire, ex. `Building (0/1)` pour un seul service avec `build`.

Conseil : plus lent qu'un build normal, à réserver aux cas où le cache donne un résultat inattendu (dépendance mise à jour non prise en compte, build qui semble ignorer une modification...).

```
docker compose down
```

Arrête et **supprime** les conteneurs et le réseau créés par `up`. Les volumes nommés sont conservés : les

données (ex. de MySQL) survivent.

Conseil : `docker compose down -v` supprime aussi les volumes, donc les données.

```
docker compose down -v --rm all
```

Nettoyage complet d'un projet Compose : supprime les conteneurs (ex. `mysql-server`), le réseau créé par Compose (nommé `<projet>_default`), les volumes (`-v`) et les images utilisées par les services (`--rm all`, ex. `mysql:latest`). Chaque élément supprimé s'affiche avec la mention `Removed`.

Conseil : `--rm local` ne supprime que les images construites par Compose (sans nom personnalisé) et garde celles téléchargées. Les messages `WARN ... variable is not set. Defaulting to a blank string` apparaissent quand les variables du fichier ne sont pas définies dans ce terminal (ex. un nouvel onglet) : sans conséquence pour un `down`, mais un `up` lancé ainsi démarrerait MySQL avec des valeurs vides.